1. Aim

To implement and analyse recursive solutions for counting the number of ways to climb a staircase when a person can take either one or two steps at a time, and to compare plain recursion, memoisation and an iterative solution.

2. Problem Explanation

A temple has n steps. A devotee can climb either one step or two steps in each move. The task is to calculate the total number of different ways to reach the top. For example, for three steps, the possible sequences are 1+1+1, 1+2 and 2+1.

The number of ways for n steps depends on the ways to reach n-1 and n-2 steps. Therefore, the recurrence relation is ways(n) = ways(n-1) + ways(n-2).

3. Base Case and Recursive Case

Base cases:

- ways(0) = 1: There is one way to climb zero remaining steps (do nothing).
- ways(1) = 1: There is only one way to climb one step.

Recursive case: for n greater than 1, ways(n) = ways(n-1) + ways(n-2). The result is the sum of the ways to reach the previous step and the step before it.

4. Answers for n = 1 to 6

Number of steps (n)	Number of ways	Example
1	1	1
2	2	1+1, 2
3	3	1+1+1, 1+2, 2+1
4	5	Five different sequences
5	8	Eight different sequences
6	13	Thirteen different sequences

5. Plain Recursive Program

```
calls = 0

def ways_naive(n):
    global calls
    calls += 1

    if n <= 1:
        return 1

    return ways_naive(n - 1) + ways_naive(n - 2)


calls = 0
print("naive:", ways_naive(5), "calls =", calls)
```

The plain recursive function directly follows the recurrence relation. However, it calculates the same smaller values many times.

6. Repeated Work – Call Tree for n = 5

The function ways_naive(5) calls ways_naive(4) and ways_naive(3). The ways_naive(4) call again calculates ways_naive(3) and ways_naive(2). The ways_naive(3) result is therefore calculated more than once.

```
ways(5)
├── ways(4)
│   ├── ways(3)
│   │   ├── ways(2)
│   │   └── ways(1)
│   └── ways(2)
└── ways(3)
    ├── ways(2)
    └── ways(1)
```

This repeated calculation makes plain recursion inefficient for large n. The number of calls grows exponentially.

7. Memoisation Using lru_cache

Memoisation stores the result of each input so that it is calculated only once. Python provides functools.lru_cache for this purpose.

```
from functools import lru_cache

calls = 0

@lru_cache(maxsize=None)
def ways_memo(n):
    global calls
    calls += 1

    if n <= 1:
        return 1

    return ways_memo(n - 1) + ways_memo(n - 2)


calls = 0
print("memo :", ways_memo(25), "calls =", calls)
```

The completed TODO uses the same base case and recursive case as the plain recursive version, but calls ways_memo instead of ways_naive. The cache prevents repeated work.

8. Complete the Code – TODO 1

```
if n <= 1:
    return 1

return ways_memo(n - 1) + ways_memo(n - 2)
```

Expected output for the assignment's n = 25 test:

```
naive: 121393 calls = 242785
memo : 121393 calls = 26
```

Both versions return the same answer. The memoised version makes far fewer function calls because each value from 0 to 25 is calculated once.

9. Function Call Growth Investigation

The following program measures the number of calls for n = 10, 20, 25 and 30. The cache must be cleared before each memoised test so that earlier results do not affect the count.

```
from functools import lru_cache

def count_naive(n):
    calls = 0

    def helper(x):
        nonlocal calls
        calls += 1
        if x <= 1:
            return 1
        return helper(x - 1) + helper(x - 2)

    answer = helper(n)
    return answer, calls


@lru_cache(maxsize=None)
def memo_helper(n):
    global memo_calls
    memo_calls += 1
    if n <= 1:
        return 1
    return memo_helper(n - 1) + memo_helper(n - 2)


for n in (10, 20, 25, 30):
    answer_naive, naive_calls = count_naive(n)

    memo_helper.cache_clear()
    memo_calls = 0
    answer_memo = memo_helper(n)

    print(n, answer_naive, naive_calls, answer_memo, memo_calls)
```

n	Naive calls	Memoised calls	Ways
10	177	11	89
20	21,891	21	10,946
25	242,785	26	121,393
30	2,692,537	31	1,346,269

The plain recursive call count grows rapidly. The memoised call count grows almost linearly because each distinct argument is evaluated once.

10. Recursion Limit for n = 1,000

Python limits the maximum recursion depth to protect the program from exhausting the call stack. The default limit can be checked using sys.getrecursionlimit(). A memoised recursive function still has recursion depth proportional to n, so ways_memo(1000) may raise RecursionError.

```
import sys
print("Recursion limit:", sys.getrecursionlimit())

try:
    print(ways_memo(1000))
except RecursionError:
    print("RecursionError: maximum recursion depth exceeded")
```

Increasing the recursion limit may allow the function to run, but setting it too high can risk exhausting the available stack. For a requirement of up to 1,000 steps, an iterative solution is safer and avoids dependence on the recursion limit.

11. Alternative Concept – Iterative Solution

A loop can calculate the answer while keeping only the previous two values. This avoids both repeated work and a large recursion stack.

```
def ways_loop(n):
    if n <= 1:
        return 1

    previous_two = 1
    previous_one = 1

    for _ in range(2, n + 1):
        current = previous_one + previous_two
        previous_two = previous_one
        previous_one = current

    return previous_one


print("loop:", ways_loop(1000))
```

The loop takes O(n) time and O(1) extra space. It is the recommended deployment choice for large staircases when only the final count is required.

12. Time and Space Complexity Analysis

Method	Time complexity	Space complexity	Explanation
Plain recursion	$O(\varphi^n)$	O(n) stack	Repeated subproblems cause exponential work; recursion depth is linear.
Memoisation	O(n)	O(n) cache and stack	Each input is calculated once and stored; recursion depth is linear.
Iterative loop	O(n)	O(1)	Each step is processed once while retaining only two previous values.

Here, φ (phi) is the golden ratio, approximately 1.618. The $O(\varphi^n)$ bound describes the exponential growth of plain recursive calls. Memoisation and iteration both require linear time, but the iterative method uses constant extra space.

13. Execution Evidence

Run the plain recursive and memoised versions for n = 10, 20, 25 and 30. Record the function-call counts from your own execution. Also run the memoised version for n = 1,000 and record the recursion limit and whether a RecursionError occurs. Run the iterative version for n = 1,000 and record its result.

- Screenshot of answers for n = 1 to 6.
- Screenshot of plain and memoised call counts.
- Screenshot of the n = 1,000 recursion-limit test.
- Screenshot of the iterative n = 1,000 result.

14. Conclusion

The staircase problem was solved using plain recursion and memoisation. Plain recursion repeats calculations and has exponential time complexity. Memoisation reduces the time to O(n), but still uses recursion stack space and may reach Python's recursion limit. The iterative solution also takes O(n) time and uses O(1) extra space, making it the best choice for calculating the number of ways for staircases up to 1,000 steps.